

POS Lab 2: SDD Pre-Video Guide

A 10-minute orientation to the process demonstrated in the video

This guide gives you the structure behind the POS Lab 2 video before you watch it. The video is not mainly about producing a Point-of-Sale application quickly. It demonstrates how a team can use Spec-Driven Development (SDD), an AI coding agent, Git branches, tests, and human review to deliver one sprint increment without losing traceability or control.

The POS project is a parallel teaching example. It demonstrates the process that students should adapt to their own TokTickIT Lab 2 requirements; it is not a solution to copy.

1. The SDD model used in this lab

SDD connects three questions:

Artifact	Main question	Scope in this lab
SRS - Software Requirements Specification	What must the application do?	The whole application: functional requirements (FR), business rules (BR), and non-functional requirements (NFR).
System-Level SDS - Software Design Specification	How will the application be built consistently?	The whole application: architecture, technology stack, shared data and API conventions, security, error handling, deployment constraints, and shared UI standards.
Feature-Level SDS / Engineering Contract	Exactly how will this feature behave and be implemented?	One bounded feature delivered through one Issue and feature branch.
STS - Software Test Specification	How will we prove the specified behavior?	Test cases for the feature, followed by sprint-level integration and system verification.

In the example repository, the Feature-Level Engineering Contract is the practical package used to control development:

Engineering Contract = Feature-Level SDS + UI/API/Data Specifications + Acceptance Criteria + STS

An engineering contract is a precise, testable promise. It defines inputs, validation, behavior, state changes, persistence, outputs, errors, and tests. Coding begins only after every implementation-affecting question has been resolved.

The System-Level SDS deliberately stops short of specifying every REST payload, Prisma model, screen interaction, and detailed test case. Those decisions belong in the relevant Feature-Level Engineering Contract, where they can be reviewed and tested within a bounded sprint scope.

System UI rules versus feature UI rules

The two levels of UI specification serve different purposes:

- docs/system/ui_spec.md is part of the System-Level SDS. It defines shared rules such as typography, colors, spacing, layout, buttons, tables, accessibility, focus behavior, and loading/error states.

- The UI section inside docs/features/<feature-id>/contract.md defines the controls, workflows, states, validation, and accessibility behavior for that particular feature.

A feature may refine the shared UI rules, but it should not silently contradict them. This prevents every feature branch from inventing a different user interface.

2. What the POS Lab 2 sprint delivers

The starting point is the completed and tested POS Lab 1 baseline on main. POS Lab 2 delivers a connected checkout increment through four bounded features:

Feature	User outcome	Dependency
Feature-D: Sale Lifecycle Management	Start, restore, and cancel an open sale.	Lab 1 baseline
Feature-E: Cart and Sale-Item Management	Scan, find, add, edit, and remove products.	Feature-D
Feature-F: VAT-Inclusive Pricing and Totals	Calculate subtotal, VAT portion, rounding, and final amount.	Feature-E
Feature-G: Order-Level Discount Management	Apply percentage or fixed discounts, validate them, and recalculate totals.	Feature-F

Together these features form the sprint increment. Each feature is also a small vertical slice: it crosses the necessary UI, API, domain logic, database, and test layers rather than implementing only one technical layer.

3. How the Markdown files organize context

The AI agent should not be expected to remember the project from an earlier chat. Durable project context is therefore kept with the code and reloaded at the start of a new session.

File or location	Scope	Purpose
AGENTS.md	Repository-wide	Defines the source-of-truth order, architecture constraints, branch lifecycle, testing rules, and working instructions for the coding agent.
docs/reference/pos-srs-features-coverage.pdf	Whole application	Authoritative SRS requirements, business rules, NFRs, and feature coverage.
docs/reference/kaching-system-level-sds-v0.1.pdf	Whole application	System architecture and cross-cutting design decisions.
docs/system/context.md	Whole application	A concise working summary for a new agent session; it does not replace the source PDFs.
docs/system/ui_spec.md	Whole application	The shared System-Level UI Design Specification.
docs/lab-02/source-evidence.md	Sprint	Separates specified facts, missing decisions, conflicts, and approved proposals, with requirement traceability.
docs/lab-02/poc-scope-and-issues.md	Sprint	Defines the selected features, exact Issue titles, scope, exclusions, acceptance criteria, dependencies, branch names, and merge order.

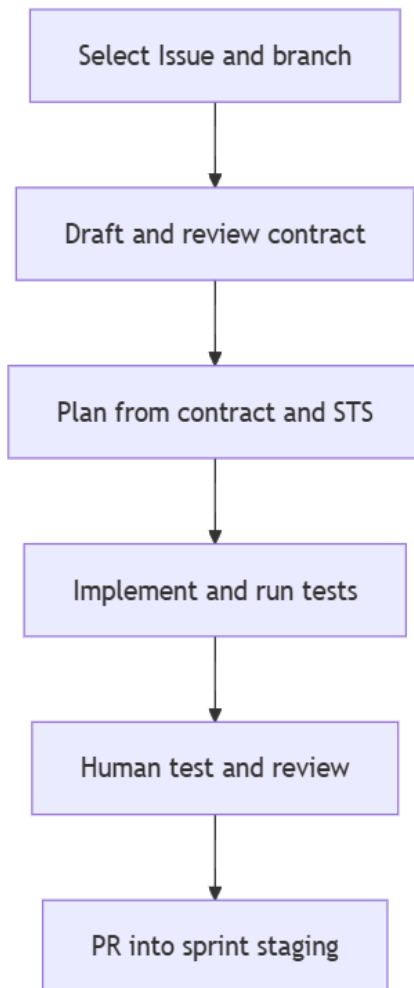
docs/templates/feature-contract.md	Reusable template	Ensures every feature contract covers the same essential sections.
docs/features/feature-d/contract.md and equivalents	One feature	The approved Feature-Level Engineering Contract containing the Feature SDS, UI/API/data design, acceptance criteria, STS, and open decisions.
client/tests/... and server/tests/...	One feature or regression area	Executable evidence that the contract has been implemented.

In this example, each feature uses one consolidated contract.md. The Feature-Level SDS, feature UI specification, API specification, data design, acceptance criteria, and STS are sections of that contract rather than several mandatory standalone files. A project may split them into multiple Markdown files when they become large, but the required content and traceability should remain the same.

The repository's source-of-truth order is important: the current lab brief and user decision come first, followed by the SRS and business rules, the System-Level SDS, the approved feature contract, and finally the existing code and tests. Existing code is not allowed to silently overrule an approved specification.

4. The lifecycle of one feature

The video shows Feature-D in detail and then repeats the same pattern for Features E, F, and G.



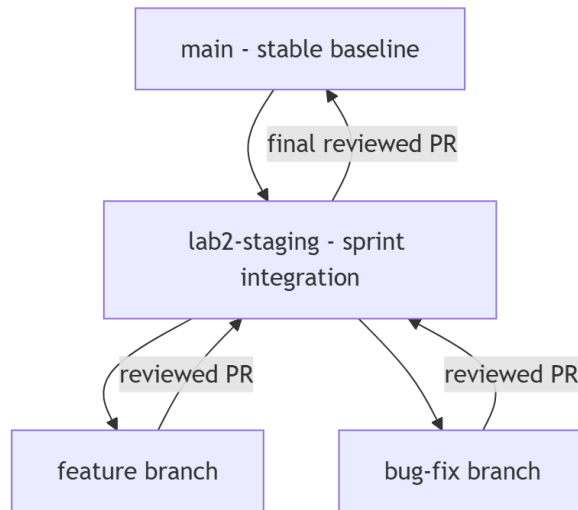
1. **Select a bounded Issue.** Confirm its requirements, scope, exclusions, dependencies, branch name, and acceptance criteria.
2. **Create the feature branch from the current sprint staging branch.** If the feature depends on an earlier feature, wait until that dependency is merged into staging, then branch from the updated staging state.
3. **Reload context.** Read AGENTS.md, the SRS and System-Level SDS evidence, sprint scope, shared UI specification, and the relevant contract template. A new AI session begins with no assumed memory.
4. **Draft the Engineering Contract before coding.** Trace every relevant FR, BR, and NFR to workflows, UI states, API behavior, data changes, acceptance criteria, and STS cases.
5. **Review the contract as a specification.** Look for ambiguity, missing failure states, unsafe transitions, accessibility gaps, untestable wording, and conflicts with higher-level sources. Resolve blocking findings and all implementation-affecting TBD items.
6. **Create an implementation plan.** Map the planned files, migrations, services, UI components, and tests to contract sections and STS IDs. This exposes missing dependencies and out-of-scope work before files are changed.
7. **Implement only the approved scope.** The agent may help write code and tests, but it should not invent requirements or expand the feature silently.
8. **Verify the result.** Run the feature tests, full regression tests, build, and application. Inspect the UI manually, exercise important failure paths, and review the Git diff against the contract. Agent self-review is useful, but it does not replace human testing or peer review.
9. **Open a PR into sprint staging.** A classmate or reviewer checks the contract, code, tests, diff, and CI result. Merge only after the review and checks pass, then update the Issue or Kanban state.

This produces a traceable chain:

SRS requirement -> Feature Contract -> Acceptance Criterion -> STS case -> Code/Test -> PR evidence

5. Branch and integration model

main is the stable release branch. lab2-staging is the integration branch for the sprint. Feature work does not merge directly into main.



The intended flow is:

1. Update and verify main.
2. Create lab2-staging from main.
3. Create each feature branch from the latest appropriate lab2-staging state.
4. Merge reviewed feature PRs into lab2-staging in dependency order.
5. Run sprint integration, regression, system, and user-oriented tests on lab2-staging.
6. If an integration bug is found, create a bugfix/... branch from lab2-staging, fix and test it there, and open a PR back into lab2-staging. Update the Feature SDS or STS when the defect reveals a specification gap.
7. Repeat integration testing until staging is acceptable.
8. Open the final PR from lab2-staging to main, obtain review and approval, then merge for deployment.

The video discovers a missing migration during integration. The general workflow above is the safer pattern for that kind of defect: keep the staging branch reviewable and apply the correction through a dedicated fix branch and PR.

6. Verification happens at different scopes

When	Verification focus	Typical evidence
On a feature branch	Does this feature satisfy its contract?	Unit tests, API/database integration tests, React component tests, build, manual UI and failure-path checks.
On sprint staging	Do all selected features work together without regression?	Migration and seed reproducibility, integrated workflow checks, regression suite, system/E2E checks, accessibility checks, and UAT evidence.
Before merging to main	Is the complete sprint increment acceptable and deployable?	Passing CI, reviewed diff, integration checklist, peer approval, and final PR.
After release	Is the deployed application operating correctly?	Production smoke checks and operational monitoring.

Passing every feature PR independently does not prove that the combined sprint works. Features can overlap, migrations can be missing, and one feature can break another. This is why the final integration audit on staging is a separate gate.

7. How the prompts support the workflow

The companion document contains prompts P1-P21. They are not a replacement for judgment; they are structured gates that change the agent's role at the right time:

- **P1-P3:** inspect evidence, define sprint scope, and resolve System-Level UI gaps before coding.
- **P4-P8:** draft, review, plan, implement, and audit Feature-D.
- **P9-P20:** reload context and repeat the contract-to-implementation cycle for Features E, F, and G.
- **P21:** audit the integrated staging branch against all acceptance criteria before recommending a merge to main.

Notice the recurring pattern: **read evidence -> specify -> review -> plan -> implement -> review -> human verify**. The agent is deliberately prevented from editing during evidence, specification-review, and planning gates. This makes its assumptions visible before they become code.

8. Video roadmap

The timestamps below are approximate and are intended as navigation aids.

Time What to watch for

- 00:00** Purpose of the POS parallel example and the end-to-end SDD workflow.
- 01:28** Verify the Lab 1 baseline: branch state, dependencies, migration, seed, tests, build, and running application.
- 04:17** Create the GitHub Project/Issues, lab2-staging, and the first feature branch.
- 07:53** Establish durable SDD context from the SRS, System-Level SDS, existing docs, and code.
- 11:38** Create the source-evidence and sprint scope documents without coding.
- 14:24** Separate fixed System-Level UI requirements from missing decisions and refine ui_spec.md after approval.
- 19:18** Begin Feature-D and create/review its Engineering Contract.
- 24:29** Plan Feature-D from the contract and STS, then implement it.
- 30:48** Review implementation against the contract; run automated and manual tests and correct defects.
- 35:58** Push Feature-D and open a reviewed PR into lab2-staging.
- 37:58** Return to staging, start a fresh session, reload context, and repeat for Feature-E.
- 56:36** Apply the same gated cycle to Features F and G in a shortened time-lapse.
- 59:12** Inspect the completed feature bundle and begin sprint integration.

62:53 Run Prompt P21 and verify the combined vertical slice on lab2-staging.

67:35 Open the final reviewed PR from staging to main.

69:54 Summary and transfer of the process to TokTickIT Lab 2.

9. What students should take away

- SDD is not "ask the AI to build the app." It is a controlled chain from requirements to evidence.
- The SRS and System-Level SDS are application-wide baselines; feature contracts are sprint-level implementation agreements.
- Shared UI standards belong at system level; feature-specific behavior belongs in the feature contract.
- A feature contract must include how the feature will be tested, not just how it will be coded.
- The AI agent should reload authoritative context at every new session and expose gaps rather than silently guessing.
- Agent-generated specifications, code, and self-reviews remain proposals until humans inspect and approve them.
- Feature completion is not sprint completion. Integration testing on staging and a final reviewed PR are separate gates.
- The recording shortens some reviews and the Feature-F/Feature-G implementation, and it does not show a fully independent peer review. In real work, do not skip that review.